



CONFIDENTIAL — INTERNAL USE ONLY

Donny AI Cost Architecture & Token Efficiency Strategy

Economic governance for Donny AI across DragonCandy's marketplace platform

Engineering

Product

Leadership

DATE

May 3, 2026

VERSION

1.0 — Approved

STATUS

**Ready for
Implementation**

For Joe, Juwan, and Dame: The Executive Summary and Sections 1–3 give you the full picture — decisions, costs, revenue impact. You can stop there.

For the dev team: Everything above, plus the Appendices at the end — schemas, API patterns, and implementation reference.

Executive Summary

Donny gets smarter and does more every month — campaign generation, creator matching, social media management, chat assistance. But every AI call costs money, and without rules, those costs grow faster than revenue. This document lays out how we keep Donny's economics healthy from pre-revenue through multi-million dollar ARR.

The rule is simple: Use the cheapest AI model that gets the job done for each task, keep usage invisible to users so they never hesitate to use Donny, and cap total AI spend at 15% of revenue so the economics get better as the business grows.

The Problem

Right now, Donny uses the expensive model for everything — including simple tasks like picking a posting time or scoring a creator, where the cheap model works just as well. That's a 12x cost difference on tasks that make up the majority of Donny's call volume. Without routing rules, costs scale with adoption and we lose control of margins.

The Solution

Three things working together: a **routing table** that sends every task to the cheapest model that handles it, an **invisible credit system** that gradually dials Donny back instead of hitting a wall, and a **revenue cap** (\$250/mo floor, 15% of revenue after that) so AI spend never outpaces the business.

Core Decisions Summary

Decision	Choice	Rationale
Model selection strategy	Task-tier routing matrix (T0–T3)	Most of what Donny does is simple and repetitive — the cheap model handles it fine
User-facing credit system	Invisible with graceful degradation	If users see a credit count, they'll hesitate to use Donny — that kills the whole point
Revenue cap enforcement	15% of revenue, \$250/mo floor	\$250/mo keeps Donny alive before we have revenue; percentage kicks in as revenue grows
Vendor strategy	Consolidate generative AI on Claude, keep OpenAI embeddings	One AI vendor bill instead of two; embeddings are pennies, not worth moving
Fine-tuning path	LoRA on open-source models at 1,000–5,000 campaigns	Once we have enough data, we can train our own cheap models and stop paying per-call

Key Stakeholder Outcome

≤15%

AI SPEND AS % OF REVENUE

≥60%

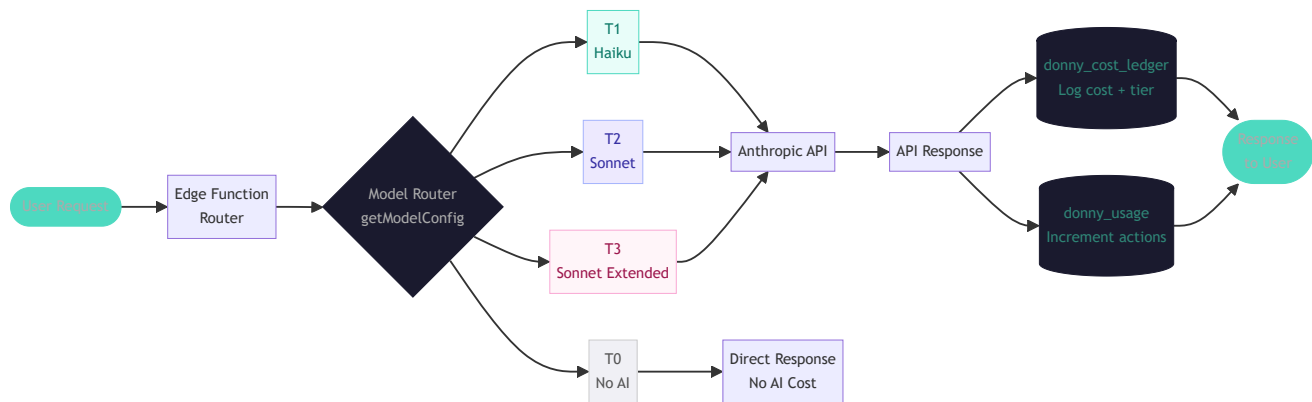
TASKS ROUTED TO HAIKU
(CHEAPEST TIER)

<5%

USERS HITTING ESSENTIAL
MODE MONTHLY

System Architecture Overview

Every Donny AI interaction flows through a deterministic pipeline. The model router is a lookup, not a judgment call — it reads the function name and usage stage, then returns the pre-configured model. Cost is logged synchronously after every API response.

END-TO-END REQUEST FLOW

Developer note: The model router is a pure function — `getModelConfig(functionName, usageStage)` — that returns a config object `{ model, maxTokens, tier }`. It has no side effects and is synchronously testable. All edge functions call this before any API invocation.

1 Model Routing Matrix

Every time Donny does something, the system checks a routing table to pick the right AI model. It's a simple lookup — not a judgment call. This means expensive models never get used for simple tasks by accident.

1.1 Tier Definitions

Tier	Model	Cost (Input / Output per MTok)	Max Tokens	When to Use
T0 No AI	None	\$0	N/A	Anything that doesn't need AI — connecting accounts, fetching data, sending already-written posts
T1 Haiku	claude-haiku-4-5	~\$0.25 / \$1.25	256–512	Simple, repetitive tasks — writing captions, picking hashtags, drafting replies, choosing posting times, formatting for different platforms
T2 Sonnet	claude-sonnet-4	~\$3 / \$15	2048–4096	Tasks that need real thinking — campaign conversations, cross-platform coordination, ROI analysis, matching creators to campaigns, checking brand guidelines
T3 Sonnet Extended	claude-sonnet-4	~\$3 / \$15	8192	Full Donny chat sessions where he's using multiple tools at once, or generating a campaign from a restaurant's URL

Why this matters: The cheap model (Haiku) costs 12x less than the smart model (Sonnet). Most of what Donny does is simple — so routing those tasks to Haiku instead of Sonnet saves real money at scale.

1.2 Edge Function Migration Table

Current production edge functions mapped to their new tier assignments. Functions already at their optimal tier are marked "No change."

Edge Function	Current Model	Current Max Tokens	New Tier	Action & Rationale
donny-chat	Sonnet 4	8192	T3	No change — this is Donny's most complex mode, needs the full model
donny-nudge-frame	Haiku 4.5	200	T1	No change — already optimally routed
donny-orchestrator	Sonnet 4	1024	T2	No change — multi-agent routing with knowledge base lookup needs the smarter model
donny-campaign-preview	Sonnet 4	4096	T1 / T2 split	Downgrade: Simple mood descriptions and storyboards drop to the cheap model. Complex multi-constraint previews stay on Sonnet. Routed by input complexity.
donny-schedule	Sonnet 4	2048	T1	Downgrade: Picking the best posting time is simple math — doesn't need the expensive model
donny-campaign-generate	GPT-4o	N/A	T2	Migrate: Moving from GPT-4o to Claude Sonnet. Campaign generation is high-value work that justifies the smarter model — and it gets us down to one AI vendor
donny-creator-match	GPT-4o	N/A	T1	Migrate + Downgrade: GPT-4o to Claude Haiku. Scoring creators against campaign criteria is straightforward — the cheap model handles it
donny-analytics-alerts	—	—	T0	Just fetching data — no AI needed

Edge Function	Current Model	Current Max Tokens	New Tier	Action & Rationale
donny- dragonshare- score	—	—	T1	Simple engagement scoring — the cheap model handles this fine
generate- embedding	OpenAI text- embedding-3- small	N/A	T0 (OpenAI)	Stays on OpenAI — costs pennies, not worth the effort to migrate

2

Invisible Credit System & Graceful Degradation

2.1 Design Principle

Users never see how much AI they're using. Period. No token counts, no credit meters, no model names. The only time a user notices anything is when they've used their whole monthly budget — and even then, it's Donny talking to them, not a system error.

Why invisible? The moment users see a credit counter, they start asking "is this worth a credit?" before every action. That kills the whole point of Donny — less typing, more doing. If users are thinking about cost, we've already failed.

2.2 The Unit: Donny Actions

Every AI call has a token cost internally. Externally, the abstraction is **Donny actions** — a normalized unit that hides the variance between a cheap Haiku caption (~800 tokens, ≈\$0.0001) and an expensive Sonnet multi-tool conversation turn (~6,000 tokens, ≈\$0.10).

Task Tier		Actions Cost	Typical Token Count	Examples
T0	No AI	0 actions	0	Scheduled post dispatch, OAuth, analytics fetch
T1	Haiku	1 action	~800 tokens	Caption generation, hashtag suggestions, nudge framing, scheduling
T2	Sonnet	3 actions	~3,000 tokens	Creator matching, campaign calendar planning, ROI analysis
T3	Sonnet Extended	5 actions	~6,000–10,000 tokens	Full Donny chat with tool use, campaign-from-URL generation
Auto-Pilot daily cycle		10 actions flat	~12,000 tokens	Full weekly content plan generation and scheduling

2.3 Tier Allocations (Monthly)

Subscription Tier	Monthly Donny Actions	Approx Token Budget	Auto-Pilot Eligible	DragonDash Rush
Free (\$0)	50	~500K tokens	No	No
Starter (\$149)	500	~5M tokens	No	Yes (+ surcharge)
Growth (\$499)	2,000	~20M tokens	Yes (standard frequency)	Yes (+ surcharge)
Pro (\$999)	10,000	~100M tokens	Yes (high frequency)	Yes (included)
Enterprise (custom)	Custom	Custom	Yes (custom schedule)	Yes (included)

2.4 Graceful Degradation — Three Stages

STAGE 1

Full Power

0–80%

- All features at designed tier
- Auto-Pilot at full frequency
- Proactive nudges active
- No user notification
- T2/T3 for complex tasks

STAGE 2

Conservation Mode

80–100%

- T2 tasks silently shift to T1
- Auto-Pilot frequency halved
- Proactive suggestions reduce
- No user notification
- On-demand still fully served

STAGE 3

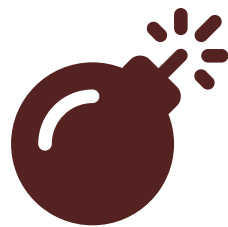
Essential Mode

100%+

- On-demand only, at T1 Haiku
- Auto-Pilot paused
- Proactive nudges stopped
- Single non-blocking message
- Overage at \$0.10–0.25/action

2.5 Degradation State Machine

USAGE STAGE TRANSITIONS



Syntax error in text
mermaid version 10.9.5

2.6 User Experience at Each Stage

The goal is that Stage 1 and Stage 2 feel identical to the user. Stage 3 is the only moment a user becomes aware of any limitation — and it is framed as Donny speaking, not a system alert.

Feature	Stage 1: Full Power	Stage 2: Conservation	Stage 3: Essential
Campaign brief generation	Sonnet (T2) — rich detail	Haiku (T1) — slightly shorter	Haiku (T1) — works
Caption suggestions	Available instantly	Available (Haiku)	Available (Haiku)
Creator matching	Haiku scoring (T1)	Haiku scoring (T1)	Haiku scoring (T1)
Auto-Pilot posting	Full weekly schedule	Every other day	Paused
Proactive nudges	Full frequency	Reduced	Stopped
What user sees	Nothing different	Nothing different	Donny: "I've been working hard this month. Upgrade to keep me at full speed, or add credits."

3

Revenue Cap Governance

3.1 The Rule

AI spend is capped at 15% of monthly revenue. Before we have revenue, the floor is \$250/month.

Same discipline as headcount — it grows with the business, not ahead of it. At \$250/mo with mostly Haiku routing, we can serve far more users than we have today.

3.2 Pre-Revenue Floor Logic

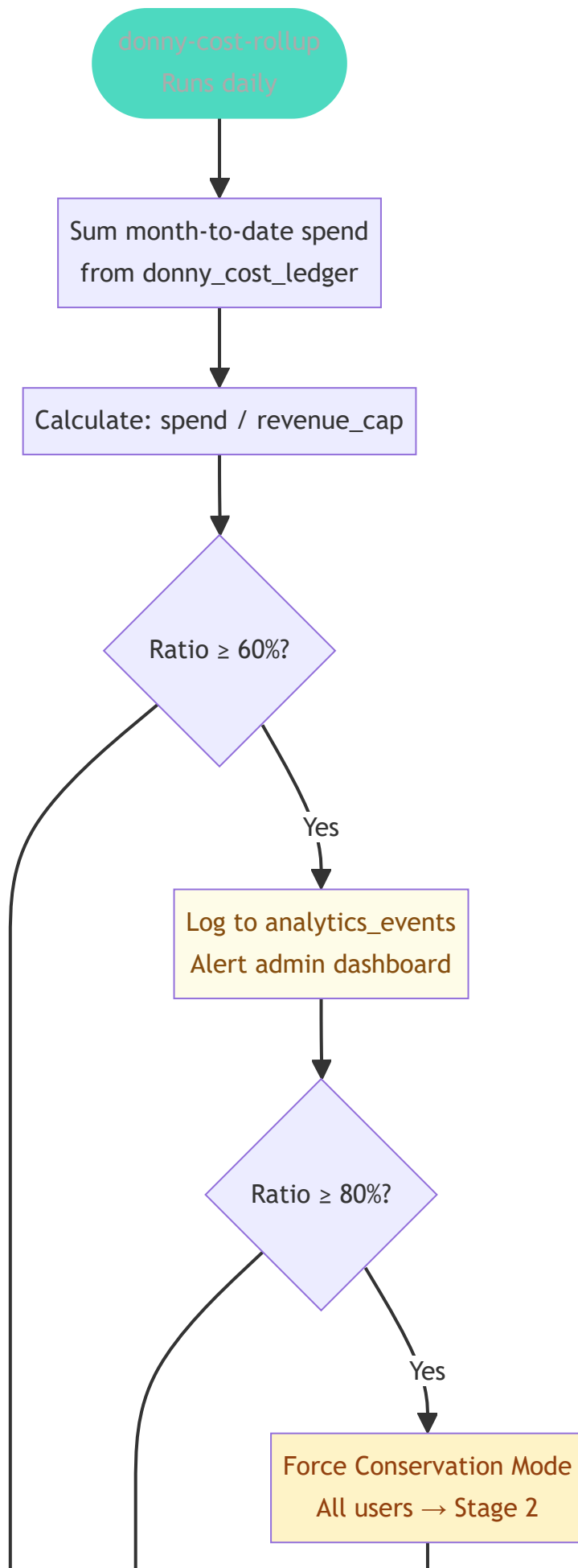
15% of \$0 is \$0, which obviously doesn't work. So we set a **\$250/month floor** that holds until revenue passes **\$1,667/month** — at that point, 15% naturally exceeds \$250 and the percentage takes over. At Haiku-dominant routing, \$250 buys roughly 50–100 million tokens per month — way more than our current ~30 users could burn through.

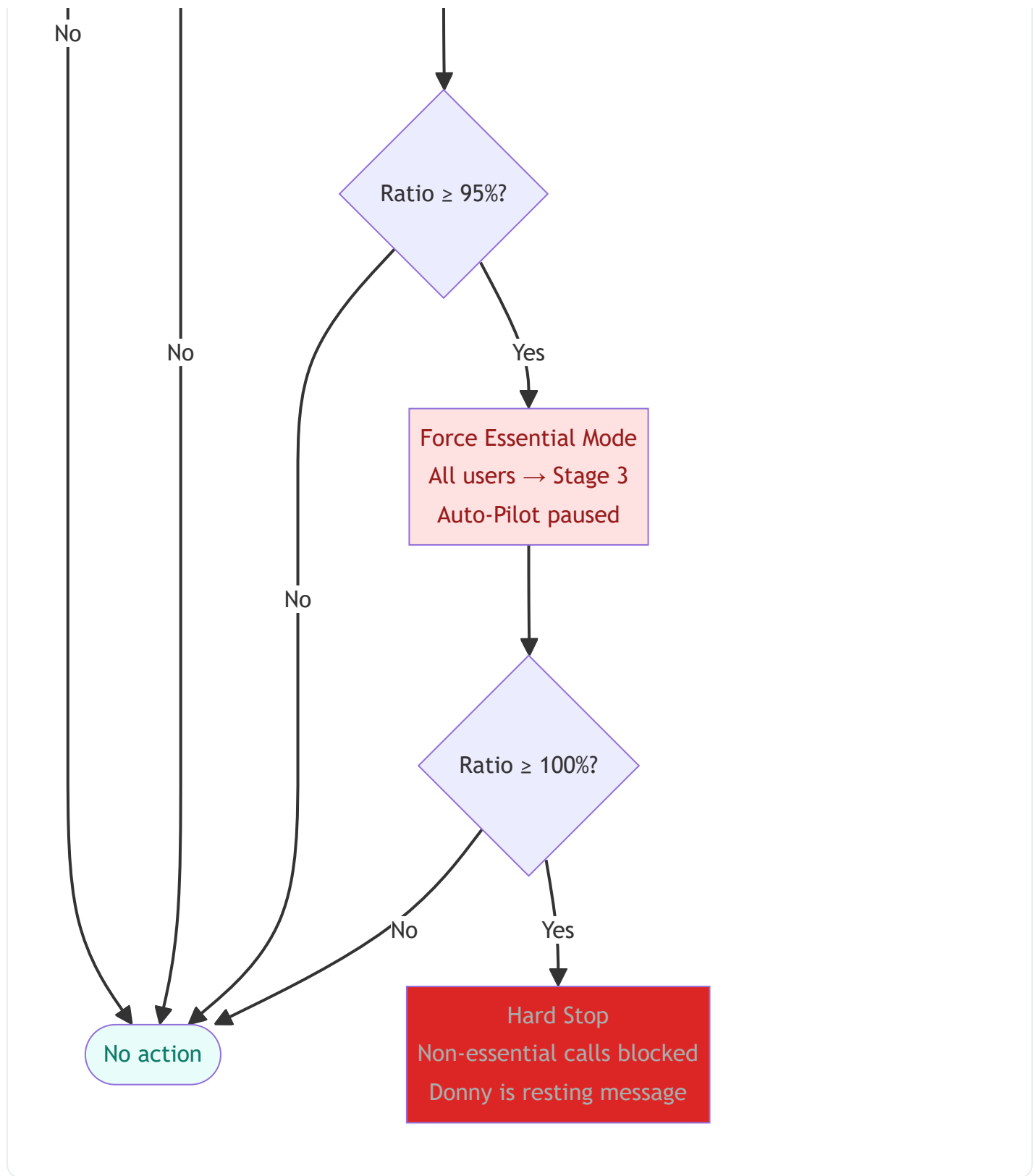
3.3 What Happens as We Approach the Cap

Threshold	Action	Who Is Affected
60% of cap	Internal alert logged to <code>analytics_events</code> , visible in admin dashboard	Engineering/ops only
80% of cap	Conservation mode forced platform-wide — all users shift to Stage 2 regardless of individual allocation	All users (silently)
95% of cap	Essential mode forced platform-wide — only on-demand T1 requests served; Auto-Pilot and proactive features paused	All users (Donny less chatty)
100% of cap	Hard stop on non-essential AI calls. Only mission-critical paths continue (campaign creation wizard, payment-related Donny actions). Everything else returns graceful "Donny is resting" message.	All users (visible, friendly)

3.4 Alert Escalation Flow

DAILY ROLLUP → ALERT ESCALATION LOGIC





3.5 Cap Scaling with Revenue

Monthly Revenue	AI Spend Cap (15%)	Per-User Budget (100 users)	Per-User Budget (1,000 users)
\$0 (pre-revenue)	\$250 floor	\$2.50	\$0.25
\$5,000	\$750	\$7.50	\$0.75
\$25,000	\$3,750	\$37.50	\$3.75
\$50,000	\$7,500	\$75.00	\$7.50
\$100,000	\$15,000	\$150.00	\$15.00

Growth insight: As revenue grows, the per-user budget grows proportionally — enabling promotion of more tasks from Haiku to Sonnet, increased Auto-Pilot frequency, and premium features unlocked at lower tiers. The model routing matrix is designed to evolve with the revenue curve.

3.6 Revenue-to-AI-Budget Visualization

\$0 revenue (floor)	\$250/mo
\$5K/mo revenue	\$750/mo
\$25K/mo revenue	\$3,750/mo
\$50K/mo revenue	\$7,500/mo
\$100K/mo revenue	\$15,000/mo

3.7 Kill-Switch Integration

If AI spend per user exceeds the revenue that user generates (LTV:AI-cost ratio < 2:1 at the individual level), that signals either tightening their tier allocation or surfacing an upgrade prompt. This mirrors the portfolio-level kill-switches from PROJECT_CONTEXT (LTV:CAC < 2:1). The `donny_cost_ledger` table provides the data for this analysis per-user and per-tier.

4

Vendor Consolidation

4.1 Short-Term (Pre-Launch, 2–4 Weeks): Migrate GPT-4o to Claude

Two of Donny's backend functions still use OpenAI's GPT-4o. We're moving both to Claude — that gets us down to one AI vendor for all generative work (we keep OpenAI just for embeddings, which cost pennies). Simpler billing, one API key to manage, and consistent routing rules across everything.

Edge Function	Current	Target	Rationale
<code>donny-campaign-generate</code>	GPT-4o	Sonnet (T2)	Structured JSON via tool use. Campaign generation is high-value output, justifies Sonnet.
<code>donny-creator-match</code>	GPT-4o	Haiku (T1)	Scoring against criteria is pattern matching — well-structured prompt with JSON tool output handles it.

Result:

One API key (Anthropic) for all generative AI. One API key (OpenAI) for embeddings only. Simpler secrets management, one billing dashboard, and consistent model governance.

4.2 Medium-Term (Post-Launch, 3–6 Months): Keep OpenAI Embeddings

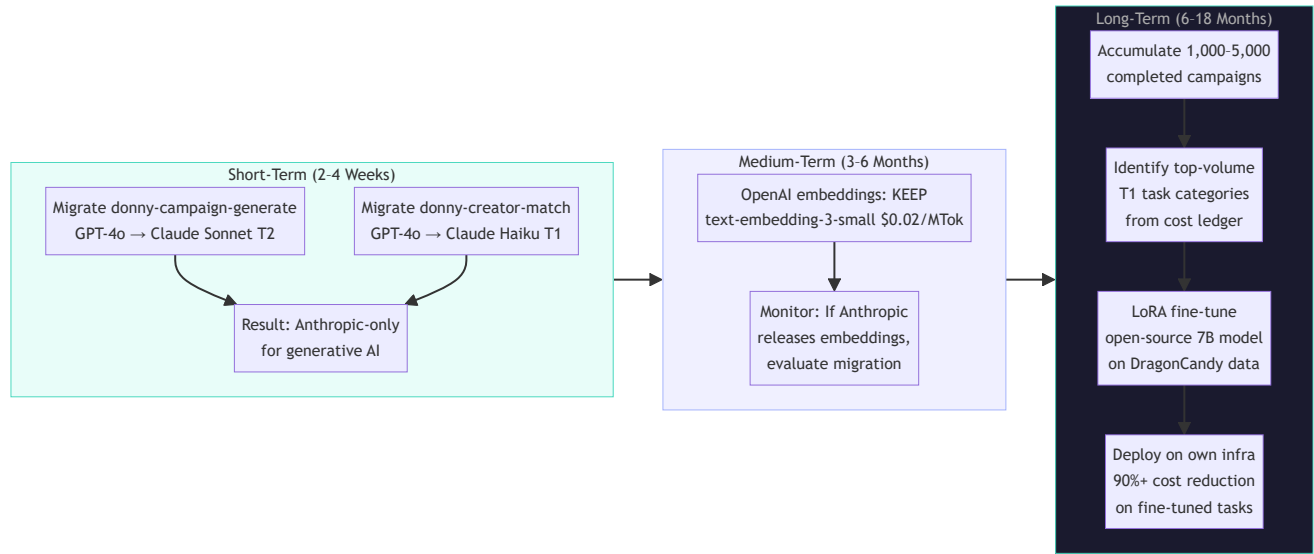
OpenAI's embedding model costs \$0.02 per million tokens — basically free. Re-indexing all our knowledge data to switch providers would cost engineering time with no real savings. The code is already isolated, so if Anthropic releases their own embedding model, it's a one-function swap.

4.3 Long-Term (1,000–5,000 Campaigns): Fine-Tuned Open-Source Models

Once we've run 1,000–5,000 campaigns, we'll have enough data to train our own small AI model for Donny's most common tasks (caption writing, hashtag generation, reply drafting). A fine-tuned model on our own infrastructure could cut per-call cost by 90%+ for these tasks. The cost ledger we're building now is what makes that decision evidence-based instead of a guess.

4.4 Consolidation Roadmap

VENDOR CONSOLIDATION TIMELINE



5

Social Media Integration Alignment

The social media integration uses the same routing rules. Every social media AI call goes through the same model selection table — no special cases, no exceptions. Here's how social features map to tiers.

5.1

DragonDash Rush Posting Differentiation

Rush multi-platform posting is always a DragonDash event — the most expensive AI action generates the most revenue. When a creator's deliverable is approved, the business sees:

Option	Tier	Action Cost	Revenue Generated	AI Work
"Post now to all platforms"	DragonDash rush	3 actions (T2)	\$25–50 surcharge	Simultaneous multi-platform posting with Sonnet-written platform-specific captions, optimized hashtags, cross-tagging
"Schedule for optimal times"	Standard	1 action (T1)	No surcharge	Haiku picks best time per platform, queues posts
"Post to one platform now"	Standard	1 action (T1)	No surcharge	Single-platform post with Haiku caption
"Edit first" / "Skip"	Free	0 actions (T0)	No surcharge	No AI — user writes manually

5.2

Model Routing Per Social Media Feature

6 of 15 social media capabilities are T1 (Haiku), 6 are T2 (Sonnet), and 3 are T0 (no AI). Versus naively assigning Sonnet to every social feature — this routing cuts social media AI cost roughly in half.

Social Media Feature	Tier	Rationale
Caption / hashtag generation	T1 Haiku	Pattern matching — templates + trending data
Engagement hub reply drafting	T1 Haiku	Short-form responses from templates
Content calendar slot suggestions	T1 Haiku	Time-slot optimization from engagement data
UGC detection & reshare prompts	T1 Haiku	Tag/mention detection + templated prompt
Google Business sync posts	T1 Haiku	Format adaptation for GBP — pattern matching
Cross-post caption rewriting	T1 Haiku	Voice/tone adaptation — pattern matching
Growth insights & recommendations	T2 Sonnet	Multi-signal analysis across platforms
Multi-platform simultaneous posting	T2 Sonnet	Complex orchestration across platform APIs
Brand guidelines enforcement	T2 Sonnet	Content review against multiple constraints
Sponsorship ROI report generation	T2 Sonnet	Multi-party analytics synthesis
Auto-Pilot weekly planner	T2 Sonnet	Multi-day content strategy generation
Sponsorship intelligence	T2 Sonnet	Cross-campaign pattern analysis
Scheduled post dispatch	T0 No AI	Pre-written content, just an API call
Analytics fetching from Outstand	T0 No AI	Data retrieval only, no generation
Account connection / OAuth	T0 No AI	Authentication flow, no AI involved

5.3 Automation Levels Tied to Subscription Tiers

Automation Level	Available On	Action Cost	Behavior
Manual	All tiers (Free+)	0 actions	User writes captions, picks times, publishes through UI. Donny helps only when explicitly asked.
Assisted (default)	Starter+ (\$149)	1–3 actions per post	Donny drafts captions and suggests optimal times. User reviews before publish. Haiku-powered.
Auto-Pilot	Growth+ (\$499)	10 actions/day flat	Donny generates weekly content plan, schedules and publishes autonomously, sends daily summary digest.

Free-tier social access: Free users still get social media features — manual mode through the UI with occasional Donny help up to their 50-action monthly limit. This keeps the integration valuable at every tier while reserving expensive automation for paying customers.

6 Success Metrics

All metrics are derived from the `donny_cost_ledger` and `donny_usage` tables. The `donny-cost-rollup` edge function runs daily and surfaces these to an admin dashboard.

Metric	Target	Measurement	Business Signal
AI spend as % of revenue	≤ 15% (or ≤ \$250/mo pre-revenue)	donny_cost_ledger monthly rollup vs. Stripe revenue	Primary health metric — above 15% triggers review
Haiku task ratio	≥ 60% of all Donny actions at T1	donny_cost_ledger tier distribution query	Routing efficiency — lower means Sonnet overuse
Fallback rate per category	< 15% per task category per 7 days	donny_cost_ledger fallback flag, grouped by edge_function	Quality signal — above 15% triggers permanent tier promotion
Stage 3 triggers	< 5% of active users per month	donny_usage where current_stage = 'essential'	UX health — above 5% means allocations are too tight
Upgrade conversion from Stage 3	> 20% upgrade within 7 days of hitting Stage 3	Subscription events correlated with Stage 3 timestamps	Monetization effectiveness of degradation design
GPT-4o elimination	0 GPT-4o calls within 4 weeks of migration	donny_cost_ledger model distribution	Vendor consolidation completion
Per-user AI cost trend	Decreasing month-over-month as Haiku routing expands	donny_cost_ledger per-user aggregation, month-over-month delta	Efficiency improvement — should trend down as routing matures
DragonDash AI cost vs. surcharge revenue	Surcharge revenue ≥ 5× AI cost of rush requests	DragonDash orders correlated with T2 cost in ledger	Confirms high-cost AI actions generate matching revenue

Key Stakeholder KPIs

≤15%

≥60%

<15%

AI COST / REVENUE CAP
\$250/mo floor pre-revenue

HAIKU (T1) TASK RATIO
Routing efficiency target

FALLBACK RATE PER CATEGORY
Quality vs. cost balance

<5%

USERS HITTING ESSENTIAL
MODE

UX health signal

>20%

STAGE 3 → UPGRADE
CONVERSION

Monetization effectiveness

0

GPT-4O CALLS POST-
MIGRATION

Vendor consolidation target

7 Guiding Principles

CHEAPEST ACCEPTABLE MODEL

Start with Haiku. Only use Sonnet when the task genuinely needs it. "It feels safer" is not a reason to use the expensive model.

INVISIBLE ECONOMICS

Users experience Donny's intelligence, not his price tag. No meters, no credit counts, no model names anywhere in the UI.

GRACEFUL OVER ABRUPT

When usage is high, Donny gets gradually quieter — he doesn't suddenly break. Conservation before essential mode before hard stop.

LEDGER-FIRST

Every AI call gets logged with its cost before the response goes back to the user. No decisions without data.

DRAGONDASH CAPTURES PREMIUM

The most expensive AI work (rush posting, complex orchestration) generates the most revenue through DragonDash surcharges. Expensive features pay for themselves.

REVENUE-ADAPTIVE

As revenue grows, we can afford to upgrade task tiers. What runs on Haiku today might graduate to Sonnet at \$50K MRR. The system is built for that.

VENDOR-LIGHT

One vendor for generative AI, one for embeddings. Fine-tuning on open-source models is how we eventually stop paying per-call entirely.

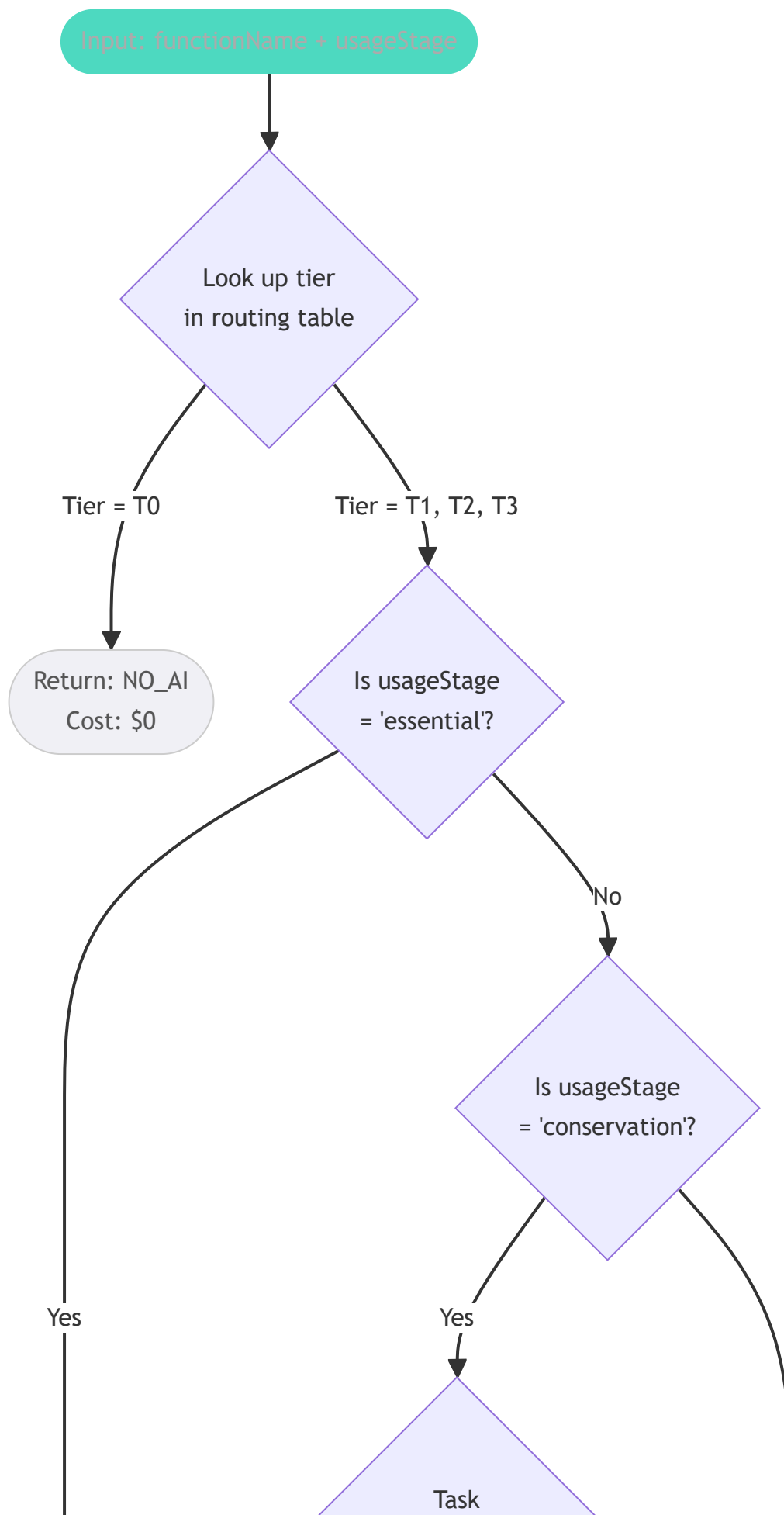
Appendices — Developer Reference

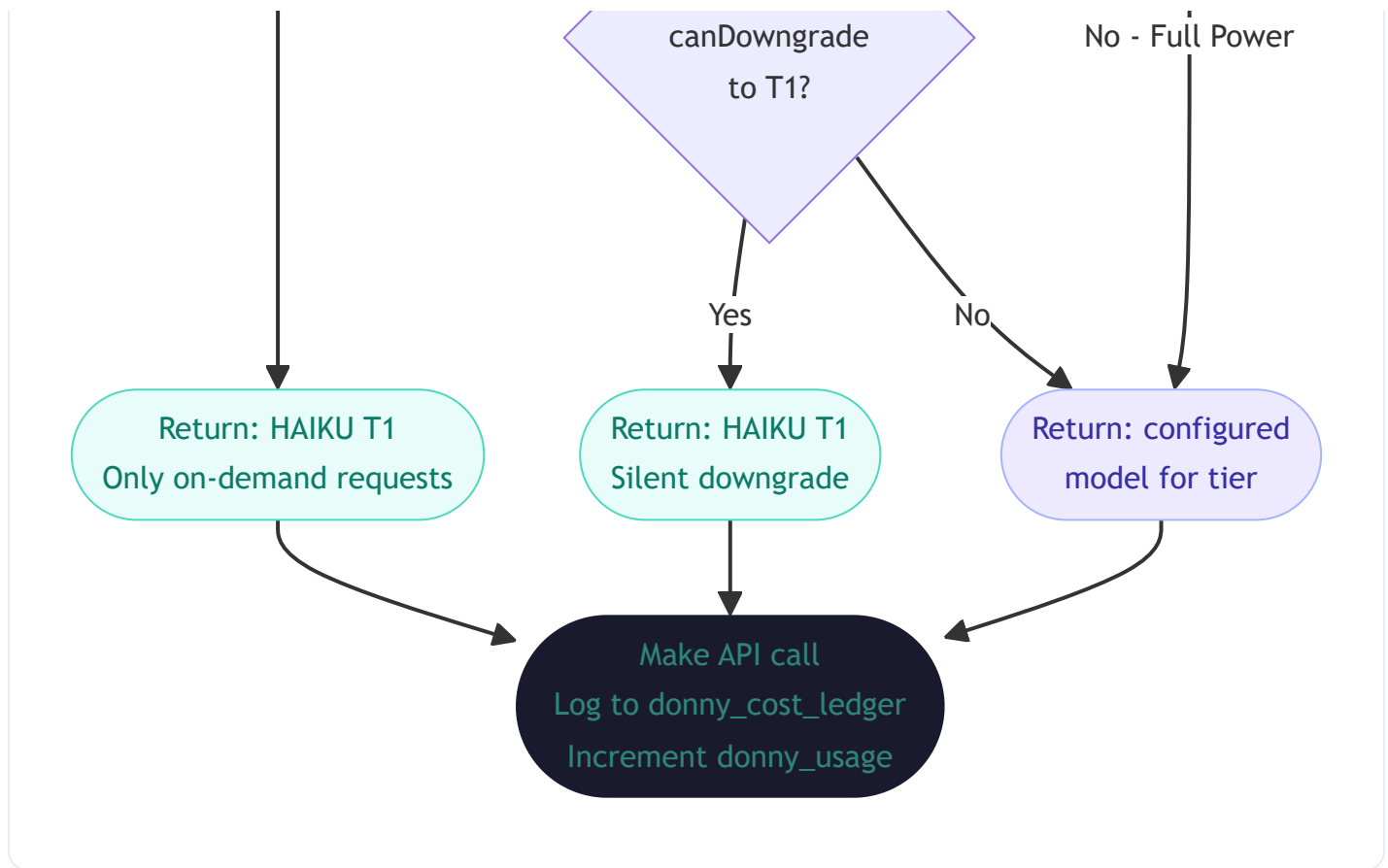
The sections below are for the dev team. They contain database schemas, routing logic diagrams, code patterns, and migration checklists. If you're reading this as a stakeholder, you can stop here — the main document covered everything you need.

Appendix A: Model Routing Logic

Model Selection Decision Flow

GETMODELCONFIG() LOGIC — DEVELOPER REFERENCE





Fallback Rule

If a Haiku response fails quality checks (malformed JSON, off-topic output, truncated response), the system retries once at Sonnet and logs the fallback to `donny_cost_ledger` with `fallback: true`.

Automatic tier promotion: If a task category triggers fallback more than 15% of the time over a rolling 7-day window, that category is permanently promoted to the next tier. Tier promotions are reviewed monthly and only kept if the fallback pattern persists. This prevents Haiku degradation from silently inflating costs.

Appendix B: Database Schema

Two new tables, following the ledger-first architecture principle already embedded in the codebase. Schema and RLS are defined before any routing code. Every AI call is logged synchronously before returning a response.

`donny_cost_ledger` — Per-Call Cost Tracking

Column	Type	Purpose
id	uuid (PK)	Primary key — gen_random_uuid()
user_id	uuid (FK → profiles)	Which user triggered the call
edge_function	text	Which edge function made the call (e.g., "donny-orchestrator")
model	text	Model used: "claude-haiku-4-5", "claude-sonnet-4", or "none"
tier	text	T0 / T1 / T2 / T3
input_tokens	integer	Tokens in the request (from Anthropic usage object)
output_tokens	integer	Tokens in the response (from Anthropic usage object)
estimated_cost_usd	numeric(10,6)	Calculated: (input_tokens × input_rate) + (output_tokens × output_rate)
fallback	boolean	True if this was a tier-promotion retry (Haiku → Sonnet fallback)
created_at	timestamptz	Call timestamp — default now()

RLS policy: Admin-only read access. Edge functions write via service role. No user-level read access (users never see their cost data directly).

Indexes: (user_id, created_at DESC) , (edge_function, tier, created_at) , (fallback, created_at) for fallback rate analysis.

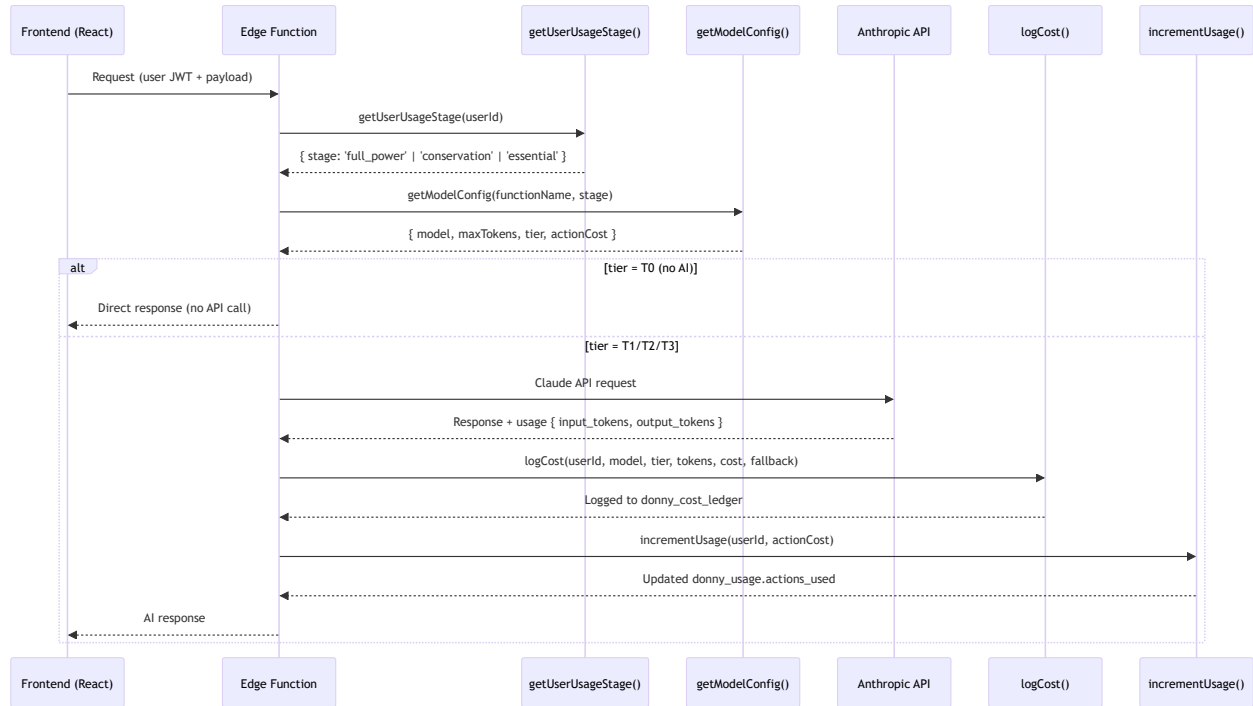
donny_usage — Per-User Action Tracking

Column	Type	Purpose
id	uuid (PK)	Primary key
user_id	uuid (FK → profiles)	One row per user per billing period
period_start	date	First day of billing period (resets monthly)
actions_used	integer	Running total of Donny actions consumed this period
actions_budget	integer	Monthly allocation for user's current subscription tier
current_stage	text	full_power / conservation / essential — drives model routing decisions
updated_at	timestampz	Last update timestamp

RLS policy: Users can read their own row (but only `current_stage` is ever surfaced in UI). Edge functions update via service role using atomic `actions_used = actions_used + N` to avoid race conditions.

Unique constraint: (`user_id`, `period_start`) — one row per user per period.

Data Flow Diagram

API CALL → COST LOG → USAGE UPDATE

Appendix C: Edge Function Integration Pattern

Every edge function that makes an AI call follows this standard pattern. This is a pseudocode reference — the actual TypeScript implementation lives in each function's `index.ts`. The key invariant: **cost is always logged, usage is always incremented, and model selection is always via the routing table, never hardcoded.**

Standard Donny AI Edge Function Pattern

```

// 1. Auth – always first
const user = await authenticateRequest(req);
if (!user) return unauthorizedResponse();

// 2. Usage stage – read before any AI call
const usageStage = await getUserUsageStage(user.id);
// Returns: 'full_power' | 'conservation' | 'essential'

// 3. Model config – lookup, not judgment
const modelConfig = getModelConfig(functionName, usageStage);
// Returns: { model, maxTokens, tier, actionCost, canMakeCall }
  
```

```
// If tier = T0, returns NO_AI config and caller skips API call

if (modelConfig.tier === 'T0') {
  // No AI call needed - execute directly
  return directResponse(payload);
}

// 4. Build the prompt (function-specific logic here)
const messages = buildMessages(payload, context);

// 5. Call the API (Anthropic only, for T1/T2/T3)
const response = await anthropic.messages.create({
  model: modelConfig.model,
  max_tokens: modelConfig.maxTokens,
  messages,
});

// 6. Quality check - for T1 tasks that might need T2 fallback
if (modelConfig.tier === 'T1' && !isValidResponse(response)) {
  const fallbackConfig = getModelConfig(functionName, usageStage, { forceTier: 'T2' }
  const fallbackResponse = await anthropic.messages.create({ ... });
  await logCost({ ... fallbackConfig, fallback: true, usage: fallbackResponse.usage
  await incrementUsage(user.id, fallbackConfig.actionCost);
  return formatResponse(fallbackResponse);
}

// 7. Log cost - synchronous, before returning
await logCost({
  userId: user.id,
  edgeFunction: functionName,
  model: modelConfig.model,
  tier: modelConfig.tier,
  inputTokens: response.usage.input_tokens,
  outputTokens: response.usage.output_tokens,
  estimatedCostUsd: calculateCost(modelConfig, response.usage),
  fallback: false,
});

// 8. Increment usage - synchronous, before returning
await incrementUsage(user.id, modelConfig.actionCost);

// 9. Return the response
return formatResponse(response);
```

Key Function Signatures

Function	Signature	Location
<code>getModelConfig</code>	<code>(functionName: string, stage: UsageStage, opts?: { forceTier?: Tier }) → ModelConfig</code>	<code>supabase/functions/_shared/modelRouter.ts</code>
<code>getUserUsageStage</code>	<code>(userId: string) → Promise<'full_power' 'conservation' 'essential'></code>	<code>supabase/functions/_shared/usageStage.ts</code>
<code>logCost</code>	<code>(params: CostLogParams) → Promise<void></code>	<code>supabase/functions/_shared/costLogger.ts</code>
<code>incrementUsage</code>	<code>(userId: string, actionCost: number) → Promise<void></code>	<code>supabase/functions/_shared/usageTracker.ts</code>
<code>calculateCost</code>	<code>(config: ModelConfig, usage: AnthropicUsage) → number</code>	<code>supabase/functions/_shared/costCalculator.ts</code>

Shared utilities location: All routing, logging, and usage utilities live in `supabase/functions/_shared/` — the existing shared module already used by `donny-chat` for CORS headers and Supabase client initialization. New utilities extend this module without altering existing patterns.

Migration Checklist for Existing Functions

1. Add `getUserUsageStage(user.id)` call immediately after auth
2. Replace hardcoded model string with `getModelConfig(functionName, stage).model`
3. Replace hardcoded `max_tokens` with `modelConfig.maxTokens`
4. Add `logCost()` call after API response, before return
5. Add `incrementUsage()` call after `logCost()`
6. Add quality check + fallback retry for T1 functions that output structured data
7. Test with `npx tsc --noEmit` then `npm run build`
8. Verify new rows appear in `donny_cost_ledger` and `donny_usage` after test call

DragonCandy — Confidential — Internal Use Only

Donny AI Cost Architecture v1.0 — May 3, 2026

Prepared by DragonCandy Engineering & Product